Documentation du code

Capteur porté par le joueur — Raspberry Pi Pico 2 W, centrale inertielle BNO055 et GPS GY-NEO6MV2 (u-blox NEO-6M), programmés en MicroPython. Modes foot (capteur au mollet) et rugby (capteur au col du maillot).

Sommaire

- 1. Vue d'ensemble et architecture
- 2. Matériel et câblage
- 3. Installation et démarrage rapide
- 4. Principes de mesure
- 5. Indicateurs du cahier des charges
- 6. Assistance GPS
- 7. Étalonnage
- 8. Référence des fichiers et fonctions
- 9. Paramètres réglables
- 10. Données produites (état, résumé, fichiers)
- 11. Traçabilité et robustesse
- 12. Limites connues
- 13. Dépannage

1. Vue d'ensemble et architecture

Le code est séparé en fichiers « capteur » (accès au matériel, un fichier par capteur) et en fichiers « traitement » (calculs). Les deux modes partagent le même tronc commun pour la mesure du déplacement.

Fichier	Type	Rôle	Dépend de
`bno055_base.py`	Pilote	Pilote de base du BNO055 (Peter Hinch, dépôt micropython-bno055)	—
`bno055.py`	Pilote	Pilote complet du BNO055 (Peter Hinch)	bno055_base
`capteur_bno055.py`	Capteur	Accès au BNO055 : réglages, lectures, acquisition cadencée, biais du gyroscope, enregistrement, calibration	bno055
`capteur_gps.py`	Capteur	Accès au GPS : lecture NMEA générique, qualité, position locale, configuration u-blox	—
`deplacement.py`	Traitement	Tronc commun : verticale, détection des cycles, modèle de cycle, navigation, indicateurs, assistance GPS	capteur_bno055
`mode_foot.py`	Traitement	Mode foot : frappes + course par foulées	deplacement
`mode_rugby.py`	Traitement	Mode rugby : course par pas	deplacement
`journal.py`	Traçabilité	Horodatage, noms de fichiers uniques, journal d'état, enregistrement brut autour des frappes	—
`main.py`	Programme	Lancé seul à la mise sous tension, pour les séances sur batterie	mode_foot, mode_rugby

Les deux modes

	Mode foot	Mode rugby
Emplacement	Mollet / tibia (solidaire de l'os)	Dos, col du maillot (haut de la colonne)
Mesures	Tirs, Tir max + course (tous les indicateurs de déplacement)	Course (tous les indicateurs de déplacement)

	Mode foot	Mode rugby
Cycle détecté	Foulée (2 pas) : pic de vitesse angulaire de la jambe	Pas : pic d'accélération verticale du tronc
Modèle de distance	$v = K \times \omega_{pic}$; distance = v × durée	$L = K \times (a_{max} - a_{min})^{1/4}$ (Weinberg)
Fréquence	125 Hz (le BNO055 limite à ≈ 210 lectures/s)	100 Hz
Réglage du BNO055	`passer_en_mode_foot()`	`passer_en_mode_rugby()`
Programme	`mode_foot.demo_foot()`	`mode_rugby.demo_rugby()`
Fichier d'étalonnage	`modele_foot.json`	`modele_rugby.json`
GPS (optionnel)	Assiste la course, pas les frappes	Assiste la course

2. Matériel et câblage

Composant	Broche du module	Pico 2 W	Broche physique
BNO055 (breakout Adafruit)	VIN	3V3(OUT)	36
	GND	GND	38
	SDA	GP4 (I2C0 SDA)	6
	SCL	GP5 (I2C0 SCL)	7
	ADR	non connecté → adresse 0x28 (relié à 3,3 V → 0x29)	—
GPS GY-NEO6MV2	VCC	3V3(OUT) (vérifier la plage admise par le module)	36
	GND	GND	38
	TX	GP1 (UART0 RX)	2
	RX	GP0 (UART0 TX)	1

Liaison	Réglages par défaut
I2C0 (BNO055)	400 kHz ; descendre à 100 kHz en cas d'OSError. La vitesse réelle est imposée par le BNO055 : ≈ 0,8 ms + 0,22 ms par octet, soit ≈ 4,7 ms par lecture complète (≈ 210 lectures/s)
UART0 (GPS)	9600 bauds ; tampon de réception 1024 octets ; jusqu'à 5 positions/s après configuration

3. Installation et démarrage rapide

- Firmware : dans Thonny, Outils → Options → Interpréteur → MicroPython (Raspberry Pi Pico) → Installer ou mettre à jour MicroPython (Pico branché avec BOOTSEL enfoncé), variante Pico 2 W.
- Fichiers : Affichage → Fichiers ; sélectionner les 8 fichiers dans « Cet ordinateur » ; clic droit → Téléverser vers /. Ou, depuis un terminal du PC :

```
mpremote cp bno055_base.py bno055.py capteur_bno055.py capteur_gps.py :
mpremote cp deplacement.py journal.py mode_foot.py mode_rugby.py :
```

Action	Commande (REPL)
Étalonner (une fois par joueur et par mode)	`import mode_rugby; mode_rugby.demo_etalonnage("course", 100)` `import mode_foot; mode_foot.demo_etalonnage("course", 100)`
Séance rugby, sans GPS	`mode_rugby.demo_rugby(600)`
Séance rugby, GPS en secours	`mode_rugby.demo_rugby(600, avec_gps=True)`
Séance foot (L = 0,50 m)	`mode_foot.demo_foot(600, longueur_levier_m=0.50)`
Séance foot avec GPS	`mode_foot.demo_foot(600, avec_gps=True)`
Tester le GPS seul	`import capteur_gps; capteur_gps.demo_gps(60)`

Action	Commande (REPL)
Enregistrer des données brutes (CSV)	`import capteur_bno055; capteur_bno055.demo_enregistrement(10)`
Orientation par fusion du BNO055	`capteur_bno055.demo_orientation(30)`

> Au lancement d'une séance, le joueur reste **immobile** pendant environ 1 s : le programme mesure le biais du gyroscope. S'il bouge, une erreur « Mouvement detecte » est levée : recommencer.

4. Principes de mesure

4.1 Chaîne de traitement (deplacement.py)

Étape	Classe	Ce qu'elle fait
1	EstimateurVertical	Suit la verticale dans le repère du capteur → accélération verticale a_{vert} et rotation autour de la verticale. Le boîtier peut être fixé dans n'importe quelle orientation.
2	NavigationPDR	Intègre le cap et avance la position d'une distance de cycle (navigation à l'estime).
3	DetecteurCycles	Détecte les pas (rugby, sur a_{vert}) ou les foulées (foot, sur $\|\omega\|$).
4	ModeleWeinberg / ModeleFoulee	Distance parcourue pendant le cycle, avec un K par allure.
5	SeanceDeplacement	Vitesse = distance des 2 derniers cycles / durée des 2 derniers cycles ; vitesse nulle sans cycle pendant 2 intervalles ; choix centrale / GPS.
6	IndicateursSeance	Indicateurs du cahier des charges à partir de la vitesse.

Pourquoi ne pas intégrer l'accélération ?

Un biais de seulement 0,05 m/s² intégré pendant une minute donne 0,05 × 60 = 3 m/s d'erreur de vitesse. L'intégration directe est donc inutilisable ; la méthode par cycles (podomètre) ne dérive pas.

4.2 Verticale (filtre complémentaire)

Un vecteur fixe dans le repère terrestre tourne dans le repère du capteur selon **du/dt = u × ω** : le gyroscope prédit la verticale u. L'accéléromètre la corrige : en moyenne il mesure +g vers le haut, donc u est rapproché de a/|a| avec une constante de temps de 2 s, seulement si 0,3 g < |a| < 3 g (phases de vol et chocs rejetées). On en déduit :

- $a_{vert} = a \cdot u - g$: accélération verticale sans la gravité (m/s², positive vers le haut) ;
- lacet = $\omega \cdot u$: rotation autour de la verticale (rad/s, positive dans le sens trigonométrique vu de dessus).

4.3 Détection des cycles

Le signal est filtré (passe-bas du 1er ordre). Un cycle est une zone au-dessus d'un seuil adaptatif max(seuil_min, fraction × enveloppe des pics récents), datée à son maximum. Deux pics trop proches sont fusionnés ; après un long arrêt, le cycle suivant est un cycle de « départ » dont la durée est inconnue.

Réglage	Rugby (pas)	Foot (foulées)
Signal	a_{vert} (m/s²)	$\|\omega\|$ de la jambe (°/s)
Passe-bas	8 Hz	6 Hz
Seuil minimal	1,5 m/s²	150 °/s
Fraction de l'enveloppe	0,3	0,5
Intervalle entre cycles	0,18 à 1,2 s	0,35 à 2,0 s
Grandeur transmise au modèle	amplitude pic − creux	valeur du pic

4.4 Modèles de distance et allures

Chaque modèle a trois coefficients K, un par allure. L'allure est choisie selon la cadence (cycles par seconde).

	ModeleWeinberg (rugby)	ModeleFoulee (foot)
Formule	$L = K \times \text{amplitude}^{1/4}$	$v = K \times \omega_{pic}$ (rad/s) ; distance = v × durée
Marche	< 2,3 pas/s ; K = 0,45	< 1,15 foulée/s ; K = 0,23
Course	2,3 à 3,4 pas/s ; K = 0,60	1,15 à 1,7 foulée/s ; K = 0,31
Sprint	≥ 3,4 pas/s ; K = 0,80	≥ 1,7 foulée/s ; K = 0,36
Cycle de départ	course si amplitude ≥ 15 m/s², sinon marche	course si ω ≥ 450 °/s, sinon marche ; durée 1,1 / 0,72 / 0,5 s

	ModeleWeinberg (rugby)	ModeleFoulee (foot)
Bornes	L entre 0,2 et 2,8 m ; K entre 0,1 et 2,0	Distance entre 0,4 et 5,5 m ; K entre 0,05 et 1,0

> Les K par défaut sont des **ordres de grandeur**. Sans étalonnage (section 7), les vitesses et distances sont fausses.

4.5 Frappes (mode foot)

La jambe (genou → cheville) est un solide en rotation autour du genou : ω est la même en tout point du tibia, quelle que soit la hauteur du capteur. La vitesse du pied vaut $\mathbf{v_{pied} = \omega_{max} \times L}$, avec L la distance genou → point de contact pied/ballon (à mesurer sur le joueur ; ce n'est pas la distance genou → capteur). Le modèle suppose la cuisse quasi immobile à l'impact.

DetecteurFrappe : un événement commence quand $|\omega| \geq 400$ °/s et finit quand $|\omega| < 200$ °/s ; il est validé si sa durée est entre 30 et 500 ms et si $|\omega|_{max} \geq 1000$ °/s (puis temps mort de 400 ms). La vitesse du ballon est estimée par $v_{ballon} \approx 1{,}2 \times v_{pied}$ (rapport à calibrer au radar).

L (genou → contact)	0,45 m	0,50 m	0,55 m
Vitesse du pied maximale mesurable (gyroscope à 2000 °/s = 34,9 rad/s)	56,5 km/h	62,8 km/h	69,1 km/h

Au-delà, la vitesse renvoyée est une borne inférieure (drapeau **vitesse_minoree**, affichage « ≥ »). L'accéléromètre (16 g) peut aussi saturer : l'intensité « acc_max_g » est alors indicative. Une foulée qui coïncide (±0,3 s) avec une frappe validée est écartée du calcul de la course.

5. Indicateurs du cahier des charges

Indicateur	Définition dans le code	Paramètre (défaut)
Tirs	Nombre de frappes validées (mode foot uniquement)	omega_min_dps (1000 °/s)
Tir max	Vitesse du pied de la frappe la plus rapide ; vitesse du ballon estimée × 1,2	longueur_levier_m (0,50 m), rapport (1,2)
Distance	Somme des distances (cycles, ou vitesse GPS × durée)	—
Activité	Durée en course / durée totale × 100	seuil_course_kmh (7 km/h)
Sprint max	Vitesse la plus élevée de la séance	—
Course	Temps total à vitesse ≥ seuil de course, en min:s	seuil_course_kmh (7 km/h)
Sprints	Nombre de « courses » (≥ 1 s au-dessus du seuil de course) dont la vitesse max ≥ 1,3 × la vitesse moyenne **en course** (définition figée au cahier des charges)	facteur_sprint (1,3), duree_min_course_s (1 s), reference_sprint ("course")
Accélérations	Dérivée de la vitesse sur 0,5 s ≥ +2 m/s² ; un seul comptage par franchissement	seuil_acceleration_ms2 (2), fenetre_acceleration_s (0,5)
Décélérations	Idem avec ≤ −2 m/s²	idem
Position	Navigation à l'estime ; recalée sur le GPS s'il est présent	—

Pour un sprint, la « vitesse moyenne » de référence est par défaut la vitesse moyenne pendant les phases de course (reference_sprint="course"). Le choix "seance" (distance / durée totale) donne un seuil plus bas, donc plus de sprints : à trancher selon la définition voulue.

6. Assistance GPS

Le GPS est optionnel et s'utilise de la même façon dans les deux modes (paramètre gps=, ou avec_gps=True dans les programmes). Il assiste uniquement la mesure du **déplacement** : une frappe dure environ 0,1 s et ne concerne que le pied, le GPS ne peut pas la mesurer.

Situation	priorite="imu" (défaut)	priorite="gps"
GPS exploitable, cycles détectés	Vitesse de la centrale ; le GPS étalonne K et le cap	Vitesse Doppler du GPS
GPS exploitable, aucun cycle et GPS ≥ 1,5 m/s	GPS en secours (source « gps (secours) »)	Vitesse Doppler du GPS
GPS absent ou mauvais	Vitesse de la centrale	Vitesse de la centrale (relais)
Position	Recalée sur le GPS à chaque mesure exploitable	Idem

Règle	Valeur par défaut
GPS exploitable si	trame RMC valide de moins de 1,5 s, qualité ≥ 1, ≥ 5 satellites, HDOP ≤ 2,5
Vitesse GPS considérée nulle	< 0,3 m/s (bruit Doppler à l'arrêt)
Étalonnage automatique (K et cap)	si vitesse GPS ≥ 2,5 m/s et |accélération| < 0,5 m/s² ; K ← K × (1 + 0,05 × (v_{GPS} / v_{cycles} − 1)), rapport limité à]0,5 ; 2[
Cap	moyenne circulaire de l'écart entre cap GPS et cap intégré (gain 0,05)

Changement de modèle de GPS

La lecture n'utilise que les trames NMEA standard RMC et GGA, tous préfixes acceptés (GP, GN, GL…). Seule configurer_ublox() est propre aux puces u-blox ; pour un autre fabricant, passer configuration_ublox=False ou écrire la fonction équivalente. Une autre classe de récepteur peut remplacer RecepteurGPS si elle fournit : mettre_a_jour(), est_exploitable(), origine_definie(), definir_origine(), position_locale(), vitesse_ms et cap_deg.

7. Étalonnage

Étape	Rugby	Foot
1. Marche, 50 m mesurés	`mode_rugby.demo_etalonnage("marche", 50)`	`mode_foot.demo_etalonnage("marche", 50)`
2. Course régulière, 100 m	`mode_rugby.demo_etalonnage("course", 100)`	`mode_foot.demo_etalonnage("course", 100)`
3. Sprint lancé, 40 m	`mode_rugby.demo_etalonnage("sprint", 40)`	`mode_foot.demo_etalonnage("sprint", 40)`

- Parcourir la distance en ligne droite à l'allure voulue, puis rester immobile 3 s : la mesure s'arrête seule.
- K[allure] est multiplié par (distance réelle − distance des autres allures) / distance estimée à cette allure : respecter l'ordre marche → course → sprint.
- Le modèle est enregistré (modele_rugby.json ou modele_foot.json) et rechargé à chaque séance ; un fichier d'un mode est refusé par l'autre mode.
- Avec le GPS, K continue d'être ajusté pendant les séances et le modèle est réenregistré à la fin.

8. Référence des fichiers et fonctions

capteur_bno055.py — fonctions

Élément	Rôle et retour
`creer_bus_i2c(id_bus=0, broche_sda=4, broche_scl=5, frequence_hz=400000)`	Crée le bus I2C matériel. Retour : machine.I2C.
`norme3(x, y, z)`	Norme d'un vecteur 3D.
`ms2_vers_g(a) ; dps_vers_rads(w) ; ms_vers_kmh(v)`	Conversions d'unités.
`sauvegarder_csv(donnees, nb, nom_fichier, biais_gyro=(0, 0, 0))`	Écrit un enregistrement en CSV : t_us, ax, ay, az (m/s²), gx, gy, gz (°/s).
`demo_orientation(duree_s=30, avec_magnetometre=True)`	Calibration de la fusion puis affichage de l'orientation à 10 Hz.
`demo_enregistrement(duree_s=10, frequence_hz=125, nom_fichier="essai.csv")`	Enregistre des données brutes en mode foot puis les écrit en CSV.

capteur_bno055.py — classe CapteurBNO055

Élément	Rôle et retour
`CapteurBNO055(i2c, adresse=0x28, quartz_externe=True, transposition=(0, 1, 2), signe=(0, 0, 0), delai_demarrage_ms=1000)`	Attend le capteur, vérifie la puce, fait un reset ; laisse le capteur en NDOF. RuntimeError si absent.
`passer_en_mode_foot(bp_acc_hz=62, bp_gyro_hz=47)`	ACCGYRO ±16 g / ±2000 °/s pour 125 Hz ; relit les réglages, puis attend la première mesure (jusqu'à 0,5 s).
`passer_en_mode_rugby(bp_acc_hz=31, bp_gyro_hz=32)`	ACCGYRO ±16 g / ±2000 °/s pour 100 Hz ; même vérification et attente.
`passer_en_mode_fusion(avec_magnetometre=True)`	NDOF (absolu) ou IMUPLUS (relatif) ; accéléromètre limité à ±4 g ; attend la première mesure (jusqu'à 1,5 s).
`mode_lu()`	Mode réellement actif (5 = ACCGYRO, 8 = IMUPLUS, 12 = NDOF).
`lire_acceleration() ; lire_gyroscope()`	(x, y, z) en m/s² ; (x, y, z) en °/s, biais retranché en ACCGYRO.
`lire_acc_gyro_rapide()`	(ax, ay, az, gx, gy, gz) en une seule transaction I2C.
`lire_euler() ; lire_quaternion() ; lire_acceleration_lineaire() ; lire_gravite()`	Sorties de la fusion (RuntimeError hors fusion).
`lire_magnetometre() ; lire_temperature()`	µT (NDOF uniquement) ; °C.
`estimer_biais_gyro(nb_mesures=200, periode_ms=5, seuil_mouvement_dps=10.0)`	Biais du gyroscope, capteur immobile ; RuntimeError si mouvement.
`echantillons(frequence_hz=125, duree_s=None, brut=False, erreurs_max_consecutives=50)`	Générateur de tuples (t_us, ax, ay, az, gx, gy, gz) à fréquence fixe. Une lecture ratée est comptée dans nb_erreurs et l'acquisition continue ; l'exception n'est relancée qu'après 50 échecs d'affilée.
`mesurer_frequence_max(nb_lectures=1000)`	Fréquence de lecture maximale (Hz).
`enregistrer(duree_s, frequence_hz=125)`	(donnees, nb) bruts en RAM, 16 octets/échantillon ; MemoryError si trop long.
`etat_calibration() ; est_calibre() ; attendre_calibration(delai_max_s=120, afficher=True)`	Niveaux de calibration 0 à 3 de la fusion.
`lire_offsets() ; ecrire_offsets(donnees)`	22 octets de calibration.

Élément	Rôle et retour
`sauvegarder_calibration(nom_fichier="calib_bno055.bin") ; charger_calibration(…)`	Calibration de la fusion sur la flash.

capteur_gps.py

Les coordonnées sont stockées en entiers (1e-5 minute d'arc ≈ 1,85 cm) car les flottants du Pico, en simple précision, ne donneraient qu'environ 0,4 m de résolution sur une latitude.

Élément	Rôle et retour
`creer_uart_gps(id_uart=0, broche_tx=0, broche_rx=1, debit=9600)`	Crée la liaison série (tampon 1024 octets).
`RecepteurGPS(uart)`	Lecteur NMEA non bloquant (RMC + GGA).
`.mettre_a_jour()`	Lit et décode ; True si une trame RMC a été décodée pendant l'appel.
`.est_exploitable(hdop_max=2.5, nb_sat_min=5, age_max_ms=1500)`	Position et vitesse fiables et récentes.
`.definir_origine() ; .origine_definie() ; .position_locale()`	Repère local : (x est, y nord) en m.
`.latitude_deg() ; .longitude_deg() ; .age_ms()`	Affichage ; âge de la dernière trame RMC.
`attributs`	fix_valide, qualite_fix, nb_satellites, hdop, altitude_m, vitesse_ms, cap_deg, heure_utc, date, nb_trames_ok, nb_erreurs, lat_unites / lon_unites (entiers, 1e-5 minute d'arc).
`configurer_ublox(uart, frequence_hz=5, trames_gardees=("GGA", "RMC"), frequence_max_hz=5)`	Désactive GLL/GSA/GSV/VTG et règle la fréquence ; dict {commande : True / False / None}.
`demo_gps(duree_s=60, frequence_hz=5)`	Affiche l'état du GPS chaque seconde.

deplacement.py

Élément	Rôle et retour
`EstimateurVertical(constante_temps_s=2.0)`	.mettre_a_jour(dt, ax, ay, az, gx, gy, gz) → (a_vert, lacet).
`DetecteurCycles(frequence_coupure_hz, seuil_min, fraction_seuil, intervalle_min_s, intervalle_max_s, caracteristique="amplitude", constante_enveloppe_s=1.0)`	.mettre_a_jour(t_us, dt, signal) → None ou (t_us, caractéristique, intervalle_s ou None).
`ModeleCycle(k, cadence_course_hz, cadence_sprint_hz, distance_min_m, distance_max_m, k_min, k_max)`	Base des modèles : allure(), distance_cycle(), etalonner(), ajuster(), vers_dict(), sauvegarder(), charger().
`NavigationPDR()`	tourner(lacet, dt), avancer(distance), cap_deg() ; attributs x, y, cap_rad, decalage_cap_rad.
`IndicateursSeance(seuil_course_kmh=7.0, seuil_acceleration_ms2=2.0, fenetre_acceleration_s=0.5, facteur_sprint=1.3, duree_min_course_s=1.0, reference_sprint="course")`	.mettre_a_jour(t_us, vitesse_ms, distance_m) ; .resume() → indicateurs (section 10).

Élément	Rôle et retour
`SeanceDeplacement(detecteur, modele, indicateurs=None, gps=None, priorite="imu", decimation=1, constante_temps_vertical_s=2.0, hdop_max=2.5, nb_sat_min=5, age_max_ms=1500, vitesse_min_gps_ms=0.3, vitesse_secours_ms=1.5, vitesse_min_recalage_ms=2.5, gain_recalage=0.05)`	Base des séances : traiter(), finaliser(), etat(), resume(), temps_depuis_dernier_cycle_s().
`creer_gps(frequence_gps_hz=5, configuration_ublox=True)`	Crée et configure le RecepteurGPS.
`executer_seance(capteur, seance, frequence_hz, duree_s, periode_affichage_s=1.0, nom_resultats=None, periode_sauvegarde_s=30.0, journal_etat=None, journal_evenements=None, afficher=True, sur_evenement=None)`	Boucle d'acquisition : traite, journalise, sauvegarde le résumé toutes les 30 s et le renvoie enrichi des compteurs de qualité.
`etalonner_sur_distance(capteur, seance, frequence_hz, distance_m, allure, fichier_modele, duree_max_s=180, arret_s=3.0, nb_cycles_min=6)`	Étalonnage sur distance connue ; renvoie (ancien K, nouveau K).
`angle_pi(a) ; format_min_s(s) ; sauvegarder_json(f, d) ; charger_json(f) ; afficher_resume(r)`	Utilitaires.

journal.py

Tous les journaux se désactivent proprement en cas d'erreur d'écriture ou de manque de place, sans jamais interrompre la séance ; la raison figure dans leur champ « erreur » du résumé.

Élément	Rôle et retour
`espace_libre_octets()`	Octets libres sur la mémoire du Pico, ou None.
`horodatage()`	{"date_heure", "horloge_valide"} ; horloge_valide est False si le Pico n'a pas été relié à Thonny depuis sa mise sous tension.
`prochain_numero(cle="seance") ; nom_unique(prefixe, extension, numero)`	Compteur persistant (compteur_seances.json) et nom de fichier numéroté.
`JournalEtat(nom_fichier, periode_s=1.0, lignes_par_bloc=20, octets_min_libres=100000)`	CSV d'une ligne par seconde : est_du(), mettre_a_jour(), fermer(), resume().
`JournalEvenements(nom_fichier, frequence_hz=125, secondes_avant=2.0, secondes_apres=1.0, nb_evenements_max=40, octets_min_libres=100000)`	Tampon circulaire en RAM : ajouter(), declencher(), fermer(), resume().
`convertir_csv(nom_binaire, nom_csv, taille_fenetre=None)`	Relit un fichier d'événements en CSV ; fonctionne sur le Pico et sur PC.

mode_foot.py

Élément	Rôle et retour
`DetecteurFrappe(longueur_levier_m=0.50, seuil_debut_dps=400.0, seuil_fin_dps=200.0, omega_min_dps=1000.0, acc_min_g=0.0, duree_min_ms=30, duree_max_ms=500, periode_refractaire_ms=400)`	.mettre_a_jour(…) → None ou dict de frappe ; en_cours() ; vitesse_max_mesurable_kmh() ; nb_frappes, vitesse_pied_max_kmh, vitesse_max_minoree.

Élément	Rôle et retour
`ModeleFoulee(k_marche=0.23, k_course=0.31, k_sprint=0.36, cadence_course_hz=1.15, cadence_sprint_hz=1.7, omega_course_dps=450.0, longueur_min_m=0.4, longueur_max_m=5.5, k_min=0.05, k_max=1.0)`	Modèle de foulée (section 4.4).
`SeanceFoot(modele=None, gps=None, detecteur_frappe=None, decimation=1, fenetre_frappe_s=0.3, **options)`	traiter() renvoie le dict d'une frappe validée ou None ; options de SeanceDeplacement acceptées.
`creer_detecteur_foulees()`	DetecteurCycles réglé pour les foulées.
`estimer_vitesse_ballon_kmh(vitesse_pied_kmh, rapport=1.2)`	Vitesse du ballon estimée.
`demo_foot(duree_s=600, avec_gps=False, priorite="imu", longueur_levier_m=0.50, frequence_gps_hz=5, configuration_ublox=True, journal=True, journal_frappes=True, afficher=True)`	Séance complète : fichiers numérotés, journaux, sauvegarde toutes les 30 s. Renvoie le résumé.
`demo_etalonnage(allure, distance_m)`	Étalonnage foot (modele_foot.json).
`FREQUENCE_FOOT_HZ = 125 ; DECIMATION_COURSE = 1`	Constantes du mode.

mode_rugby.py

Élément	Rôle et retour
`ModeleWeinberg(k_marche=0.45, k_course=0.60, k_sprint=0.80, cadence_course_hz=2.3, cadence_sprint_hz=3.4, amplitude_course_ms2=15.0, longueur_min_m=0.2, longueur_max_m=2.8, k_min=0.1, k_max=2.0)`	Modèle de pas (section 4.4).
`SeanceRugby(modele=None, gps=None, **options)`	Séance rugby ; options de SeanceDeplacement acceptées.
`creer_detecteur_pas()`	DetecteurCycles réglé pour les pas.
`demo_rugby(duree_s=600, avec_gps=False, priorite="imu", frequence_gps_hz=5, configuration_ublox=True, journal=True, afficher=True)`	Séance complète : fichiers numérotés, journal d'état, sauvegarde toutes les 30 s. Renvoie le résumé.
`demo_etalonnage(allure, distance_m)`	Étalonnage rugby (modele_rugby.json).
`FREQUENCE_RUGBY_HZ = 100`	Constante du mode.

9. Paramètres réglables

Les réglages qui comptent le plus, à ajuster d'après le plan de tests :

Paramètre	Où	Défaut	Effet
longueur_levier_m	DetecteurFrappe	0,50 m	Vitesse du pied ; à mesurer sur chaque joueur
omega_min_dps	DetecteurFrappe	1000 °/s	Plus haut : moins de sprints pris pour des frappes, mais passes faibles ignorées
rapport	estimer_vitesse_ballon_kmh	1,2	Passage vitesse du pied → vitesse du ballon ; à calibrer au radar
K (3 allures)	ModeleWeinberg / ModeleFoulee	voir 4.4	Distances et vitesses ; par étalonnage
seuil_min, fraction_seuil	creer_detecteur_pas / _foulees	voir 4.3	Sensibilité de la détection des cycles
seuil_course_kmh	IndicateursSeance	7 km/h	Activité, Course, découpage des sprints
seuil_acceleration_ms2	IndicateursSeance	2 m/s²	Accélérations et décélérations
facteur_sprint	IndicateursSeance	1,3	Définition d'un sprint (+30 %)
priorite	SeanceDeplacement	"imu"	Source de vitesse prioritaire
FREQUENCE_FOOT_HZ	mode_foot.py	125 Hz	À baisser si des échantillons sont manqués (max ≈ 150 Hz)
frequence_hz (I2C)	creer_bus_i2c	400 kHz	100 kHz en cas d'erreurs I2C

10. Données produites

10.1 État instantané — etat() (pour l'affichage temps réel, FP5)

Clé	Contenu
vitesse_kmh, acceleration_ms2, distance_km	Vitesse, accélération et distance actuelles
x_m, y_m, cap_deg	Position (m) et cap (°)
nb_pas (rugby) / nb_foulees (foot), cadence_hz	Cycles détectés et cadence (cycles/s)
source	« imu », « gps » ou « gps (secours) »
gps_ok, nb_satellites, hdop	Uniquement avec GPS
tirs	Uniquement en mode foot

10.2 Résumé de séance — resume() (restitution, FP4)

Clé	Contenu
distance_km, activite_pct, sprint_max_kmh	Distance, Activité, Sprint max
course (« min:ss »), course_s	Temps passé à courir
sprints, accelerations, decelerations	Compteurs
duree_s, vitesse_moy_course_kmh, seuil_sprint_kmh	Durée, vitesse moyenne de course, seuil de sprint utilisé
nb_pas / nb_foulees, modele	Cycles détectés ; K et cadences du modèle
nb_positions_gps	Positions GPS utilisées (avec GPS)
reference_sprint	Référence du seuil de sprint ("course")
partiel	True si le résumé est une sauvegarde intermédiaire (séance interrompue)
echantillons, echantillons_manques, erreurs_lecture	Qualité de l'acquisition
horodatage	{date_heure, horloge_valide}

Clé	Contenu
journal_etat, journal_evenements	Fichier, volume, état et éventuelle erreur de chaque journal
tirs, tir_max_pied_kmh, tir_max_minore, tir_max_ballon_estime_kmh, nb_foulees_ecartees	Mode foot

10.3 Frappe validée — dict renvoyé par SeanceFoot.traiter()

numero, t_us (instant du pic), duree_ms, omega_max_dps, acc_max_g (intensité), vitesse_pied_kmh, vitesse_minoree (gyroscope saturé), acc_saturee (accéléromètre saturé).

10.4 Fichiers écrits sur la flash du Pico

Fichier	Contenu	Volume indicatif
`seance_<mode>_NNNN.json`	Résumé de la séance, réécrit toutes les 30 s	< 1 ko
`etat_<mode>_NNNN.csv`	Journal d'état, 1 ligne par seconde	≈ 2,4 ko/min
`frappes_NNNN.bin`	Brut autour de chaque frappe (mode foot)	6 ko par frappe
`compteur_seances.json`	Compteur de séances par mode	quelques octets
`modele_rugby.json`	K et cadences du modèle de pas	< 1 ko
`modele_foot.json`	K et cadences du modèle de foulée	< 1 ko
`calib_bno055.bin`	Calibration de la fusion	22 octets
`essai.csv`	Acquisition brute continue (demo_enregistrement)	≈ 50 o/ligne

NNNN s'incrémente à chaque séance : aucune séance n'écrase la précédente. Récupération dans Thonny (panneau Fichiers, clic droit → Télécharger) ou depuis un terminal du PC :

```
mpremote cp :seance_foot_0007.json .
mpremote cp :frappes_0007.bin .
```

11. Traçabilité et robustesse

Les seuils de détection ne sont pas encore réglés : sans trace de ce qui a été mesuré, chaque sortie terrain dont les seuils se révèlent mauvais est une sortie à refaire. Trois niveaux de trace répondent à ce besoin, du plus léger au plus lourd.

Trace	Contenu	Volume	À quoi elle sert
Résumé JSON	Indicateurs, réécrits toutes les 30 s	< 1 ko	Résultat de la séance ; une coupure ne fait perdre que la dernière demi-minute
Journal d'état	Vitesse, accélération, distance, position, source ; 1 ligne/s	≈ 2,4 ko/min	Rejouer la chronologie, tracer la courbe de vitesse, vérifier les bascules centrale/GPS
Journal d'événements	Brut à 125 Hz, 2 s avant et 1 s après chaque frappe	6 ko par frappe	Régler après coup omega_min_dps et les autres seuils, sans retourner sur le terrain

Pourquoi pas une acquisition brute continue

À 125 Hz, 16 octets par échantillon font 2 ko/s, soit environ 10,8 Mo pour 90 min : bien plus que la mémoire libre du Pico. Le brut n'est donc conservé qu'autour des événements. Pour une acquisition brute continue et courte, utiliser `CapteurBNO055.enregistrer()`.

Tolérance aux pannes

Situation	Comportement
Lecture I2C ratée	3 relances internes ; en cas d'échec l'échantillon est abandonné, compté dans erreurs_lecture, et l'acquisition continue
Capteur débranché	L'exception n'est relancée qu'après 50 échecs d'affilée (paramètre erreurs_max_consecutives)
Mémoire flash pleine	Le journal concerné s'arrête proprement et note la raison dans le résumé ; la séance continue
Écriture impossible	Idem : aucune exception n'est propagée depuis un journal

Situation	Comportement
Coupure d'alimentation	Le dernier résumé enregistré reste exploitable, avec partiel=True

Horloge

Le Pico n'a pas de pile d'horloge : après une coupure, la date repart à l'origine du système. Thonny règle l'horloge quand il se connecte, donc la date reste juste tant que le Pico n'a pas été débranché. Le champ `horloge_valide` du résumé le signale, et le numéro de séance (compteur persistant) reste fiable dans tous les cas.

Relire un journal d'événements

```
import journal
journal.convertir_csv("frappes_0007.bin", "frappes_0007.csv", 375)
```

Le nombre d'échantillons par événement (375 par défaut) figure dans le résumé de séance, champ `journal_evenements.echantillons_par_evenement`. La fonction marche aussi sur un PC.

12. Limites connues

Limite	Conséquence	Piste
Gyroscope limité à 2000 °/s	Vitesse du pied plafonnée (62,8 km/h pour L = 0,50 m) ; frappes les plus fortes minorées	Gyroscope ±4000 °/s
Modèles de distance à étalonner	Vitesses fausses sans étalonnage ; précision dégradée lors des changements d'allure	Étalonnage par joueur, GPS
Mouvements non vers l'avant	Pas chassés, en arrière, mêlée : comptés comme des pas vers l'avant	GPS
Contacts et sauts	Faux pas possibles	Seuils de détection
Position à l'estime	Dérive avec le temps sans GPS	GPS
Foulées de sprint (mode foot)	Peuvent dépasser 1000 °/s et être prises pour des frappes	Régler omega_min_dps sur des enregistrements
Lecture I2C lente du BNO055	≈ 210 lectures complètes/s au maximum : mode foot limité à 125 Hz	Lire le gyroscope seul (≈ 495/s) si les pics de frappe sont sous-estimés
v_pied = ω × L suppose le genou immobile	En course le genou avance : la vitesse de frappe est sous-estimée de 10 à 25 %	Intégrer l'accélération sur la durée du geste
Mémoire flash limitée	Pas d'enregistrement brut continu sur une séance entière	Brut autour des frappes seulement (section 11)
GPS	Aucune mesure en intérieur ; 1 à 5 mesures/s ; aucune aide pour la frappe	—
Base fixe C0 et liaison radio	Non programmées (FP3, FP4, FP5)	etat() et resume() prêts à transmettre

13. Dépannage

Symptôme	Cause probable	Solution
RuntimeError « BNO055 introuvable »	Câblage I2C, alimentation, adresse	Vérifier SDA = GP4, SCL = GP5, 3V3 et GND ; scanner le bus
OSError [Errno 5] EIO	Faux contact (barrettes non soudées, fils Dupont usés)	Souder, raccourcir les fils ; tester la stabilité (1000 lectures)
OSError pendant les lectures	Bus I2C trop rapide (câbles longs)	`creer_bus_i2c(frequence_hz=100000)`
NameError après Run (Thonny)	Run redémarre le Pico et efface les import	Mettre les import en tête de chaque script
Fréquence max ≈ 200 Hz	Limite du BNO055 (normal)	Rien à faire ; mode foot à 125 Hz
RuntimeError « Configuration refusee »	Réglage non pris en compte par le capteur	Recommencer ; vérifier l'alimentation
RuntimeError « Mouvement detecte »	Mouvement pendant la mesure du biais	Rester immobile 1 s au démarrage
RuntimeError « … impossible en mode … »	Fonction appelée dans le mauvais mode	Appeler d'abord passer_en_mode_foot / _rugby / _fusion
MemoryError dans enregistrer()	Durée × fréquence trop grande	Réduire la durée ou la fréquence
Échantillons manqués nombreux	Traitement trop long pour la fréquence	Baisser FREQUENCE_FOOT_HZ ; limiter les print
configurer_ublox renvoie None	Pas de réponse du GPS	Vérifier GP0 → RX du GPS ; débit 9600
Aucune trame GPS (read() renvoie None)	TX/RX inversés, GPS non alimenté	GPS TX → GP1, GPS RX → GP0
gps_ok reste False	Pas de fix, HDOP élevé, < 5 satellites	Extérieur, ciel dégagé ; attendre le premier fix
Vitesses ou distances fausses	Modèle non étalonné	Étalonner (section 7)
« ATTENTION : modele … non etalonne »	Fichier de modèle absent ou d'un autre mode	Lancer demo_etalonnage du bon mode
Sprints comptés comme des tirs	omega_min_dps trop bas	Le relever d'après frappes_NNNN.bin (section 11)
erreurs_lecture > 0 dans le résumé	Faux contacts passagers	Souder, raccourcir les fils ; la séance reste exploitable
partiel=True dans le résumé	Séance interrompue (batterie, erreur)	Les données jusqu'à la dernière sauvegarde sont valides
Un journal est inactif dans le résumé	Mémoire pleine ou écriture impossible	Lire son champ erreur ; effacer les anciennes séances
Date absurde dans le résumé	Horloge non réglée (horloge_valide=False)	Normal sur batterie ; se repérer avec le numéro de séance